EdgeTrust-Offload

Secure, Dynamic Task Offloading in Edge Computing

Wanghley Soares Martins, Yifei Sun

System Architecture

- **Sensor Node:** ESP32 + MPU6050 Motion Sensing
- **Edge Cluster:** Jetson Nano + Raspberry Pi
- **Security:** Tailscale (WireGuard Zero-Trust)

Research Goal

Quantify the **Latency** vs. **Security** trade-off in real-world edge deployments

March 31, 2026 | IEEE Edge Computing Conference Format

Motivation

Low Latency Meets Zero-Trust Security

The Edge Computing Paradox

Promise of Edge

Processing data near the source to minimize latency and bandwidth usage.

Reality Check

The "Trusted LAN" assumption is obsolete. Sensitive data (motion, health, telemetry) requires protection.

Zero-Trust Cost

Mandatory encryption and authentication add significant computation and network overhead.

The Security Overhead Problem

Traditional: Data → Compute → Result

Zero-Trust Reality:

+ Network Latency & Jitter

Key Question: When does security overhead break offloading performance?

Key Idea

Local vs. Secure Offload Trade-offs

Local Compute (ESP32)

Advantages

No network transmission cost
Predictable latency (no jitter)
Data never leaves the device

Limitations

Slower CPU (240 MHz max)
Limited RAM (constrained models)
Higher energy drain for heavy math

Secure Offload (Edge Server)

Advantages

Powerful compute (Jetson GPU/CPU)
Scalable for complex algorithms
Offloads battery drain from sensor

Limitations

Network latency & variability
Security overhead (encryption/auth)
Potential for packet loss

The Core Trade-off Formula

Cost = **Tcompute** vs. **Tnetwork** vs. **Tsecurity**

Goal: Find the crossover point where offloading stops being worth it.

System Overview

Real Secure Edge Setup (Not Simulation)

Sensor Node (Client)

ESP32-S3: Dual-core 240MHz, Wi-Fi 4

MPU6050: 6-axis accelerometer + gyro

Generates 100Hz motion data

Aura Rack (Edge Cluster)

Jetson Nano: 128-core GPU (Compute)

Raspberry Pi 4 / Orange Pi: General purpose nodes

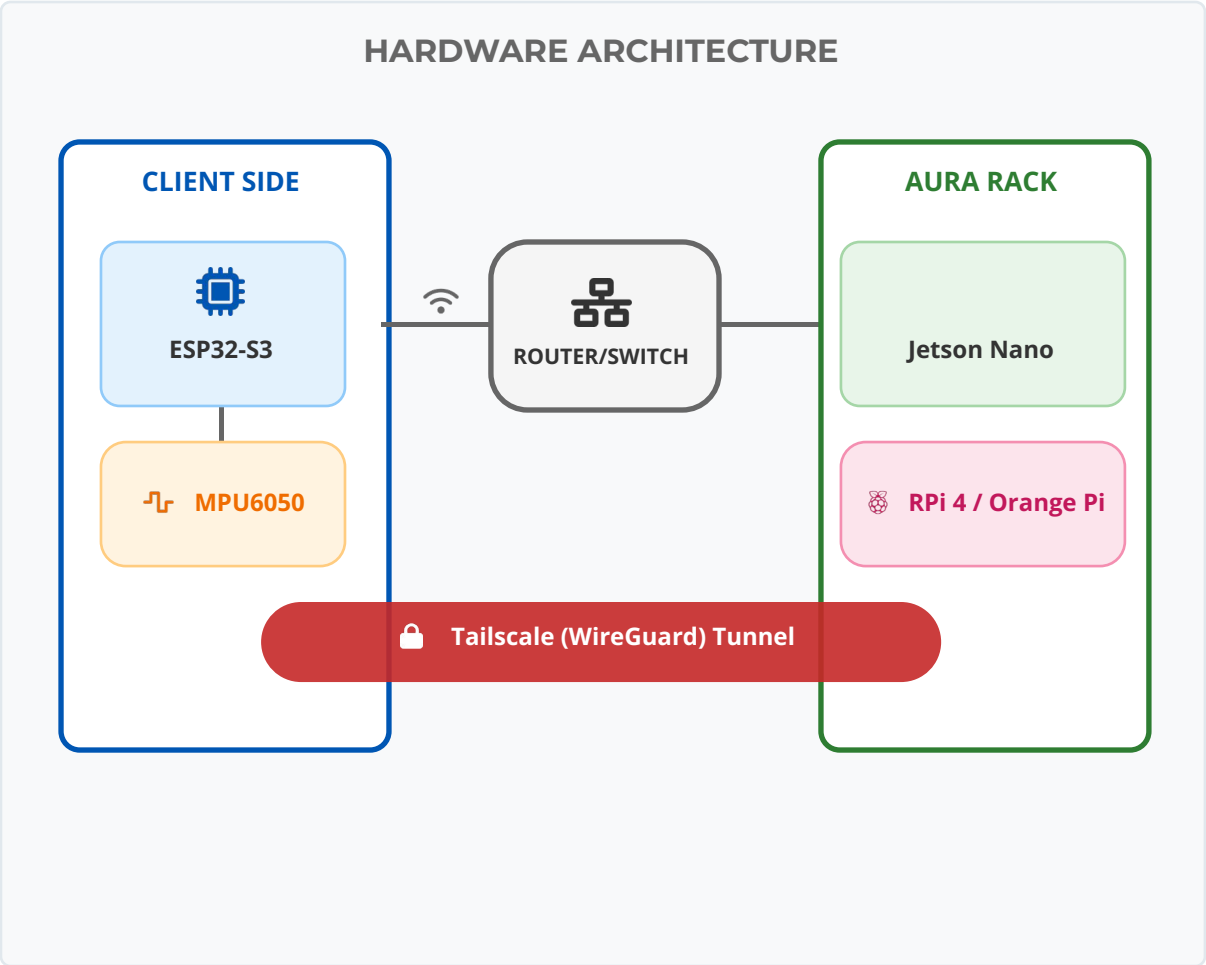
Orchestrates offloaded tasks

Network & Security

Gigabit Switch + Wi-Fi Router

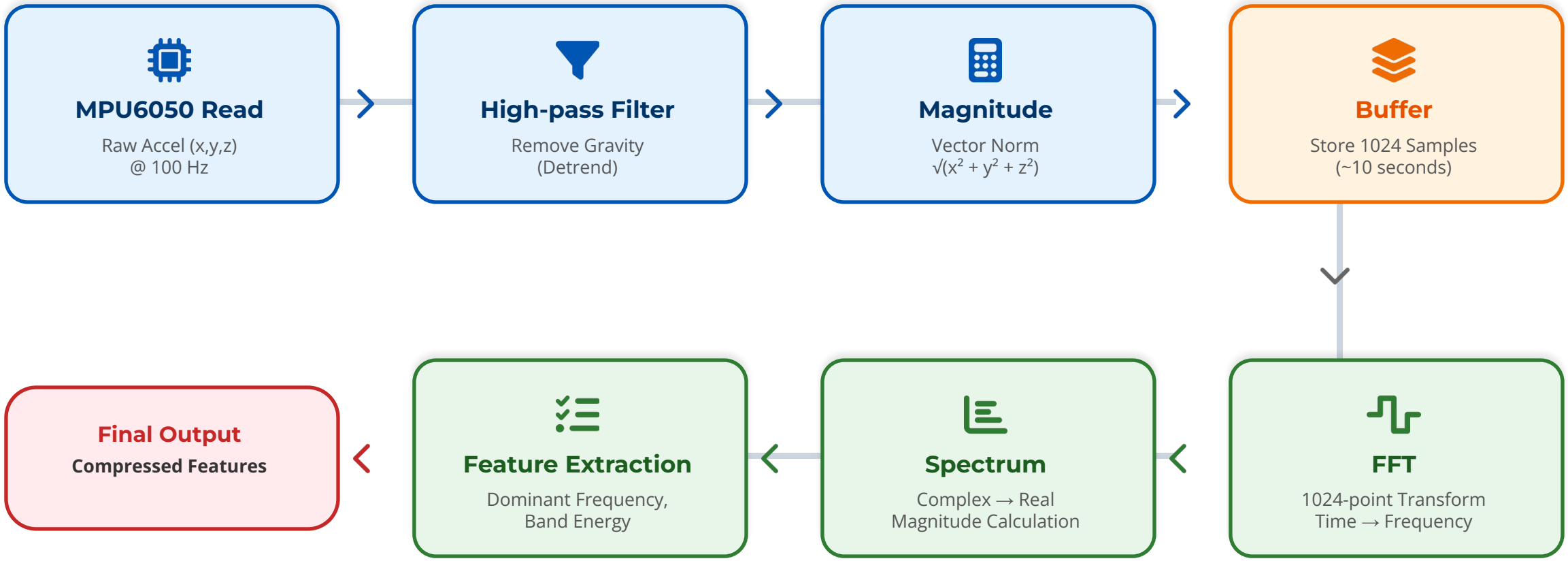
Tailscale: Zero-Trust Overlay

WireGuard Tunnel (End-to-End Encryption)



ESP32 Motion Pipeline

From Raw Data to Frequency Features (On-Device Processing)



Pipeline transforms raw time-series data (heavy) into frequency domain features (lightweight) directly on the edge node.

Why FFT for Motion?

Revealing Hidden Patterns in the Frequency Domain

From Time to Frequency

Motion = Signal Over Time

Raw accelerometer data is noisy and complex. Time-domain analysis misses subtle periodic structures.

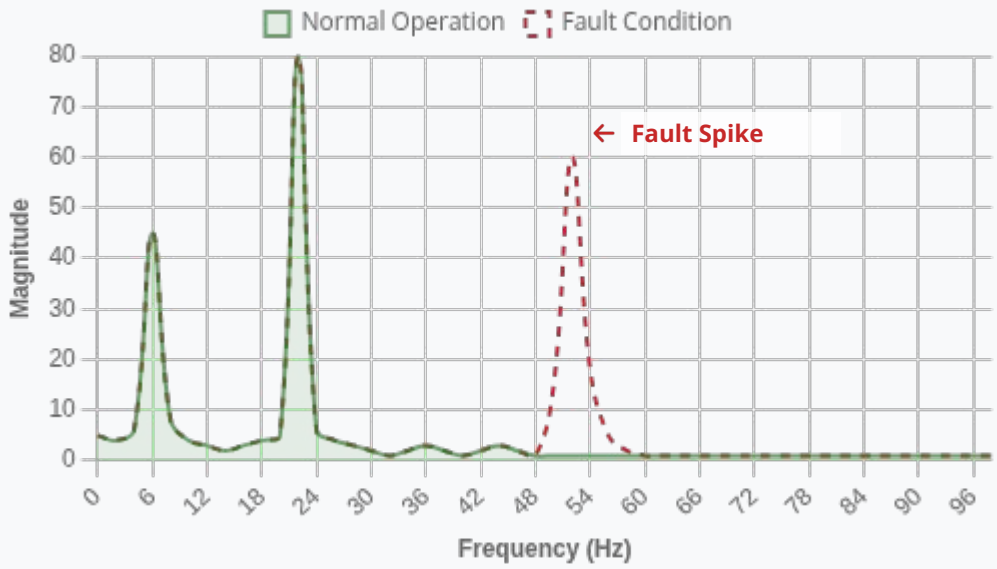
FFT Advantage

Transforms signal to frequency domain to isolate vibration patterns, motor speeds, and harmonics.

Anomaly Detection

New or spiking frequencies often indicate mechanical failure (e.g., bearing wear, imbalance).

Example: Detecting Machine Faults



Key Insight: Machine faults manifest as distinct frequency spikes

Local Performance

Establishing the Baseline (L_{local}) on ESP32-S3

Processing Latency

Per 1024-Sample Window

Stable & Predictable

Zero network jitter. Standard deviation $\sigma \approx 0$ ms.

Zero Network Delay

Data never leaves the device. T_{network} = 0.

Performance Baseline

Offloading is only beneficial if T_{total} < 30 ms.

Why Not Always Offload?

Balancing Compute Speed vs. Network Overhead

Local (ESP32)

Advantages

No network transmission delay
Stable, deterministic latency
Zero exposure to network jitter

Limitations

Limited compute headroom
Slower processing speed
Cannot handle complex ML models

Offload (Edge Server)

Advantages

Significantly faster computation
Access to GPU acceleration
Offloads energy from battery node

Limitations

Transmission time & serialization
Queueing & waiting delays
Security overhead (encryption cost)

Critical Observation

There is no "always correct" answer.

The best choice depends entirely on current network conditions and task complexity.

Experimental Design

Four Scenarios to Isolate Performance Costs

SCENARIO 1

Local (ESP32)

The Control Group

No network involvement
Pure computation time
Baseline: ~30 ms latency

SCENARIO 2

Insecure Offload

Raw TCP / UDP

Network transmission only
No encryption overhead
Goal: Measure pure network cost

SCENARIO 3

Secure Offload

Tailscale / WireGuard

Zero-Trust Overlay active
Encryption/Decryption + Auth
Goal: Isolate security overhead

SCENARIO 4

Secure + Congestion

Simulated Network Stress

Added Jitter & Packet Loss
Worst-case real-world conditions
Goal: Test system resilience

Objective: By comparing these four states, we quantify exactly *when* and *why* offloading fails.

What is Jitter?

The Enemy of Real-Time Predictability

Definition

Jitter is the variation in latency (delay) over time. Even if the *average* latency is low, high jitter means some packets arrive much later than others, disrupting the flow of real-time data.

Primary Causes

Network Congestion: Router queues filling up unpredictably.

Retransmissions: Packet loss forces TCP to resend data, adding delay spikes.

Route Changes: Packets taking different paths through the network.

The "Silent Killer" Effect

Unpredictable Delay: Deadlines are missed even if average speed is fast.

Buffer Bloat: Systems must wait longer to reorder packets.

Control Instability: Makes real-time control loops (e.g., robotics) unstable.

Metrics & Crossover Point

Quantifying the Cost of Security

Decision Threshold

Where Offload Latency $\approx$ Local (30ms)

End-to-End Latency (T_{total})

$T_{total} = T_{Tx} + T_{compute} + T_{Rx}$

Security Overhead Ratio

Additional cost of Zero-Trust vs. Plain TCP

System Throughput

Windows processed per second (Local vs. Remote)

Experimental Results

Latency Comparison & The Crossover Point

Small Tasks Favor Local

For inputs < 512 samples, local compute beats network RTT.

Security Shifts Crossover

Encryption overhead pushes the break-even point to ~1024 samples.

Large Tasks Favor Offload

GPU acceleration outperforms network costs for heavy loads (>2048).

Congestion Kills Benefit

High jitter makes offloading worse than local even for large tasks.

Duke Relevance

Real-World Applications of Secure Edge Offloading

Biomedical Sensing

Processing high-frequency motion and ECG data from wearables. Securely offloading complex anomaly detection algorithms while strictly preserving patient privacy and data confidentiality.

Smart Infrastructure

Continuous vibration and condition monitoring for bridges and industrial machinery. Enabling real-time fault detection using edge FFT features without exposing critical structural data to open networks.

Secure IoT Systems

Implementing Zero-Trust architectures directly on resource-constrained edge devices (ESP32). Moving security enforcement from the network perimeter to the device itself.

Adaptive Offload Policy

Real-world application of data-driven scheduling. Balancing latency, energy, and security costs dynamically based on live network jitter and task complexity measurements.

Future Work

The Granularity Trade-off: Accuracy vs. Bandwidth vs. Privacy

1. Raw Data

Payload: Heavy (~6 kB/window)
Pros: Highest fidelity, retrainable
Cons: High latency cost, privacy risk

2. Filtered / Decimated

Payload: Medium (~3 kB)
Pros: Less noise, moderate size
Cons: Lossy, still reveals patterns

3. Features (FFT)

Payload: Tiny (~40 bytes)
Pros: Instant Tx, High Privacy
Cons: Irreversible (cannot reconstruct)

Conclusion: Sending features saves 99% bandwidth and boosts privacy.

Limitations & Future Work

Current Constraints & The Path Forward

Current Scope

Simple Workloads Only

Limited to DSP tasks like FFT and FIR filters. Does not yet handle complex inference chains.

Approximate Energy Model

Power consumption is estimated via software models rather than direct hardware measurement.

Fixed Hardware Topology

Static 1-to-1 mapping between sensor and edge node; lacks dynamic cluster discovery.

Future Directions

3 months more - Advanced ML Workloads

Deploying CNNs and LSTMs for complex anomaly detection and predictive maintenance.

6 months more - Multi-Node Cluster Scheduling

Distributing heavy tasks across multiple Jetson/Pi nodes to balance load and heat.

6 months more - Real world deployment

Integrating with biomedical sensing applications or smart infrastructure systems.

Insignts

Summary of Contributions & Key Insights

1 Real Secure System

2 Integrating into Real system

3 Advice for future students

Final Takeaway

Security introduces a measurable cost—but by understanding the trade-offs, we can design adaptive systems that ensure privacy without sacrificing real-time performance.

Questions?

EdgeTrust-Offload Project

Adaptive Offloading in Action

Switching between **Local Compute** and **Secure Offload** to demonstrate the crossover point shift (L*) under induced network jitter.

ESP32-S3 • MPU6050 • Jetson Nano • Tailscale • WireGuard